🤖 Week 3 Assignment — RAG-Powered Console Chatbot

WnCC Machine Learning Learner Space 2026

This week's assignment has three parts of increasing difficulty.

Part A — Concept Check: Short questions testing your understanding of the week's theory. **Part B — Core Implementation:** Build a complete RAG pipeline from scratch and wire it to an LLM. **Part C — The Challenge:** Make your chatbot smarter with multi-turn memory and source citation.

Rules:

- All `# TODO` blocks must be completed. Read the docstring above each one carefully.
- Run every **Sanity Check** cell after completing a TODO. Fix failures before moving on.
- Part A answers go in the Markdown cells provided — write proper explanations, not one-liners.
- Parts B and C are graded on correctness (sanity checks), code quality, and your reflection.

Run the setup cell first.

In [1]:

```
# ========== SETUP (run this first) ==========
!pip install "transformers<5.0.0" torch faiss-cpu sentence-transformers -q

# Authenticate with HuggingFace
# Get your free token at: https://huggingface.co/settings/tokens
from huggingface_hub import login
from google.colab import userdata
HF_TOKEN = userdata.get('HF_TOKEN')
login(HF_TOKEN)

import torch
import numpy as np
import faiss
import textwrap
from transformers import pipeline

print('Setup complete!')
print(f'Device: {"GPU" if torch.cuda.is_available() else "CPU"}')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 1.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 47.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.5/18.5 MB 36.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 566.4/566.4 kB 20.0 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are inst
alled. This behaviour is the source of the following dependency conflicts.
gradio 6.19.0 requires huggingface-hub<2.0,>=1.2.0, but you have huggingface-hub 0.36.2 which is inc
ompatible.

WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels
will not work

Setup complete!
Device: CPU
```

Part A — Concept Check

Answer each question in the Markdown cell below it. **2–4 sentences each.** Be precise.

These are not trick questions — they test whether you read the material, not whether you can Google.

A1. Self-Attention

The self-attention formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

In your own words: what does the matrix QK^T represent before the softmax is applied? What does it become after the softmax? And what does multiplying by V finally produce?

Your answer:

QK^T is the multiplication of the Query Matrix with the Key Matrix that yields a matrix which shows us the compatibility score between the query of word i and the key of word j, higher the dot product value - higher is the relevance. It is divided by the square root of vectore size to shrink the variance of the softmax function down so that it does not approach values with near 0 gradient.

The softmax function then normalizes these raw probabilities into clean probabilities whose values lie between 0 and 1, it shows what focus of word i should be allotted to word j.

Multiplying by V means multiplying by the content matrix and gives us the total output of the word i by multiplying the percentage weights of each word j with it's vector to give us a weighted sum of all the words.

A2. Encoder vs Decoder

Your manager asks you to build two systems:

1. A tool that reads customer support tickets and tags them as `billing`, `technical`, or `general`.
2. A tool that automatically drafts a reply to each ticket.

Which Transformer family (encoder-only, decoder-only, or encoder-decoder) would you use for each, and why? Be specific about what architectural property makes each one suited to its task.

Your answer:

1. We would use an Encoder Transformer family (BERT) for this task as it requires classification of the ticket and tagging it into a particular category for which our tool would need to have complete context of the ticket and would require bi-directional understanding.
2. Since the ticket is already classified and passed into our tool, now we need to draft a reply to this ticket for which we would linearly need to traverse through each word and generate our draft - something that a decoder transformer family (GPT) is really good for.

A3. RLHF

Explain in your own words why RLHF uses human *comparisons* ("which of these two responses is better?") rather than human-written *ideal responses* ("write the perfect answer to this question"). What practical problem does this solve?

Your answer:

Because when asked which of the two responses (that were generated by AI) are better, we get a ranking on which we can train a completely separate small neural network called the Reward Model which would just look at an AI- predicted response and give it a score that a human would. We then hook this Reward Model into the LLM using PPO (Proximal Policy Optimization) - which based on the scores updates the internal weights of the LLM.

This solves the problem that if the LLM was generating answers that are rubbish - lower scores are assigned and during each feedback loop those weights move further away from these lower values, and if the score was high the weights are moved in a direction that continues that momentum, thus helping the model learn from the scores that a Human gave it - Reinformenet Learning from Human Feedback.

A4. RAG vs Fine-tuning

A startup has 10,000 internal company documents (policies, FAQs, product specs) and wants to build a Q&A bot over them. The documents are updated weekly.

They are deciding between: (a) fine-tuning an LLM on all the documents, or (b) building a RAG system.

Make the case for RAG. Give at least two concrete reasons why it is better suited to this scenario.

Your answer:

1. If the coduments are updated weekly we would have to fine tune our models according to the latest documents - something that would take up a lot more tokens and effort than simply providing the updated document as context each week inside the prompt - something that RAG enables us to do. 2.Fine Tuning in comparison to RAG is slow, expensive and requires a lot more effort - even after Fine Tuning the model can still hallucinate facts it learnt during fine tuning - something that would not happen with RAG since everything is provided as context in real time.

Part B — Core Implementation

You will build a **RAG-powered chatbot** in stages:

```
   Documents → Chunk → Embed → FAISS Index   (offline, build once)
                                    |
   User question → Embed → Retrieve top-k chunks → Build prompt → LLM → Answer
```

The document corpus, embedding model, and LLM pipeline are all provided. **Your job is to implement each stage of the pipeline.**

In [ ]:

```
# ========== PROVIDED: Document Corpus ==========
# A set of short ML-focused documents. Your chatbot will answer questions about these.
DOCUMENTS = [
    """The Transformer architecture was introduced in the 2017 paper 'Attention Is All You Need'
    by Vaswani et al. at Google Brain. It replaced recurrent neural networks with self-attention
    mechanisms, allowing the model to process all tokens in parallel rather than sequentially.
    This enabled training on much larger datasets and dramatically improved performance on
    sequence-to-sequence tasks like translation and summarisation.""",

    """BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only
    Transformer introduced by Google in 2018. It is trained using Masked Language Modelling,
    where 15% of input tokens are randomly masked and the model must predict them using
    both left and right context simultaneously. BERT is primarily used for text classification,
    named entity recognition, and extractive question answering.""",

    """GPT (Generative Pre-trained Transformer) is a decoder-only Transformer developed by
    OpenAI. It is trained on next-token prediction: given a sequence of tokens, predict the
    next one. GPT models use causal attention, meaning each token can only attend to previous
    tokens. This makes them naturally suited for text generation tasks. GPT-4, released in
    2023, is estimated to have around 1 trillion parameters.""",

    """RLHF (Reinforcement Learning from Human Feedback) is the training technique used to
    align language models with human preferences. It has three stages: supervised fine-tuning
    on instruction-following examples, training a reward model on human comparisons of
    model outputs, and using PPO (Proximal Policy Optimisation) to fine-tune the language
    model to maximise the reward model's scores. ChatGPT and Claude were both trained with RLHF.""",

    """RAG (Retrieval-Augmented Generation) is a technique that combines a language model with
    an external knowledge retrieval system. Documents are chunked, embedded using a sentence
    encoder, and stored in a vector database. At inference time, the user's query is embedded,
    the most similar document chunks are retrieved, and these chunks are injected into the
    LLM's prompt as context. This allows the model to answer questions about documents it was
    never trained on, without the hallucination risk of relying solely on parametric memory.""",

    """Word2Vec is a family of models introduced by Mikolov et al. at Google in 2013 for
    learning dense word embeddings. The two architectures are Skip-gram (predicts context
    words given a centre word) and CBOW (predicts a centre word given its context words).
    Words with similar meanings cluster together in the embedding space, enabling vector
    arithmetic such as: king - man + woman ≈ queen. Word2Vec embeddings are static —
    each word has one fixed vector regardless of context.""",

    """Few-shot prompting is a technique where 2-5 examples of a task are included directly
    in the prompt, allowing the LLM to infer the task format without any weight updates.
    Chain-of-Thought (CoT) prompting extends this by including step-by-step reasoning in
    the examples, which dramatically improves performance on arithmetic, logical, and
    multi-step reasoning tasks. Simply appending 'Let's think step by step' to a prompt
    enables zero-shot CoT reasoning.""",

    """Vector databases store high-dimensional embeddings and support efficient approximate
    nearest-neighbour search. Given a query vector, they return the K stored vectors with
    the highest cosine similarity or dot product. Popular options include FAISS (Facebook
    AI Similarity Search, open-source), Pinecone (managed cloud service), Chroma (lightweight,
    local), and Weaviate (open-source with a GraphQL API). FAISS uses techniques like
    product quantisation and inverted file indices to search billions of vectors in milliseconds.""",
]

print(f'Corpus: {len(DOCUMENTS)} documents loaded.')
```

```
Corpus: 8 documents loaded.
```

In [ ]:

```
# ========== PROVIDED: Models ==========
# Do NOT change these — they are fixed for the assignment.

print('Loading embedding model...')
EMBED_MODEL = 'sentence-transformers/all-MiniLM-L6-v2'
embedder = pipeline(
    'feature-extraction',
    model=EMBED_MODEL,
    tokenizer=EMBED_MODEL,
    device=-1,   # -1 = CPU; change to 0 for GPU if available
)
EMBED_DIM = 384
print(f'Embedding model loaded. Output dim: {EMBED_DIM}')

print('Loading text generation model...')
generator = pipeline(
    'text2text-generation',
    model='google/flan-t5-base',
    device=-1,
)
print('Generation model loaded.')
```

```
Loading embedding model...

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.
co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or dataset
s.
  warnings.warn(

config.json:   0%|          | 0.00/612 [00:00<?, ?B/s]

model.safetensors:   0%|          | 0.00/90.9M [00:00<?, ?B/s]

tokenizer_config.json:   0%|          | 0.00/350 [00:00<?, ?B/s]

vocab.txt: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

special_tokens_map.json:   0%|          | 0.00/112 [00:00<?, ?B/s]

Device set to use cpu

Embedding model loaded. Output dim: 384
Loading text generation model...

config.json: 0.00B [00:00, ?B/s]

model.safetensors:   0%|          | 0.00/990M [00:00<?, ?B/s]

generation_config.json:   0%|          | 0.00/147 [00:00<?, ?B/s]

tokenizer_config.json: 0.00B [00:00, ?B/s]

spiece.model:   0%|          | 0.00/792k [00:00<?, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

special_tokens_map.json: 0.00B [00:00, ?B/s]

Device set to use cpu

Generation model loaded.
```

B1 — Embed a Single Text [TODO]

The embedding pipeline returns a nested list. Your job is to:

1. Run the text through `embedder`
2. Mean-pool over the token dimension (average across all token vectors)
3. Return a 1D numpy array of shape `(EMBED_DIM,)` with dtype `float32`

In [ ]:

```python
def embed_text(text: str) -> np.ndarray:
    """
    Embed a single string into a dense vector.

    Args:
        text: input string
    Returns:
        numpy array of shape (EMBED_DIM,), dtype float32

    TODO:
      1. raw = embedder(text)             # returns a nested list: [[[f, f, f, ...], ...], ...]
      2. Convert raw[0] to a 2D numpy array of shape (num_tokens, EMBED_DIM)
      3. Mean-pool along axis 0 -> shape (EMBED_DIM,)
      4. Return as float32
    """
    # YOUR CODE HERE
    raw = embedder(text)
    tokens = np.array(raw[0])
    mean_pooled = np.mean(tokens, axis=0)
    return mean_pooled.astype(np.float32)
    pass


# ===== Sanity Check =====
vec = embed_text('Hello world')
assert vec is not None, 'embed_text() returned None'
assert isinstance(vec, np.ndarray), f'Expected np.ndarray, got {type(vec)}'
assert vec.shape == (EMBED_DIM,), f'Expected shape ({EMBED_DIM},), got {vec.shape}'
assert vec.dtype == np.float32, f'Expected float32, got {vec.dtype}'
print(f'PASS: embed_text() works! Shape: {vec.shape}')
```

```
PASS: embed_text() works! Shape: (384,)
```

B2 — Chunk Documents [TODO]

Split a list of long documents into smaller, overlapping chunks. This ensures no single chunk is too long for the model and that information at chunk boundaries is not lost.

In [ ]:

```
def chunk_documents(
    documents: list[str],
    chunk_size: int = 200,
    overlap: int = 40,
) -> list[str]:
    """
    Split each document into overlapping character-level chunks.

    Args:
        documents: list of document strings
        chunk_size: target size of each chunk in characters
        overlap: number of characters to overlap between consecutive chunks

    Returns:
        flat list of chunk strings

    TODO:
      For each document:
        Use a sliding window of size chunk_size with step (chunk_size - overlap).
        i.e., start positions: 0, chunk_size-overlap, 2*(chunk_size-overlap), ...
        Each chunk is doc[i : i + chunk_size].
        Strip whitespace from each chunk and skip empty ones.
      Return the flat list of all chunks across all documents.
    """
    # YOUR CODE HERE
    step = chunk_size - overlap
    chunks = []
    for doc in documents:
        for i in range(0, len(doc), step):
            chunk = doc[i : i + chunk_size].strip()
            if chunk:
                chunks.append(chunk)
    return chunks

    pass


# ===== Sanity Check =====
test_chunks = chunk_documents(['a' * 500], chunk_size=200, overlap=40)
assert test_chunks is not None, 'chunk_documents() returned None'
assert len(test_chunks) > 1, 'Should produce multiple chunks for a 500-char doc'
assert all(len(c) <= 200 for c in test_chunks), 'A chunk exceeded chunk_size'
print(f'PASS: chunk_documents() works! {len(test_chunks)} chunks from 500-char doc')

chunks = chunk_documents(DOCUMENTS)
print(f'Full corpus: {len(DOCUMENTS)} documents -> {len(chunks)} chunks')
print(f'Sample chunk: "{chunks[0][:120]}..."')
```

```
PASS: chunk_documents() works! 4 chunks from 500-char doc
Full corpus: 8 documents -> 27 chunks
Sample chunk: "The Transformer architecture was introduced in the 2017 paper 'Attention Is All You N
eed'
    by Vaswani et al. at Googl..."
```

B3 — Build the FAISS Index [TODO]

Embed all chunks and store them in a FAISS index for fast nearest-neighbour retrieval.

In [ ]:

```python
def build_index(chunks: list[str]) -> tuple[faiss.Index, np.ndarray]:
    """
    Embed all chunks and build a FAISS inner-product index.

    Args:
        chunks: list of text chunks
    Returns:
        (index, embeddings) where:
          - index is a faiss.IndexFlatIP ready for search
          - embeddings is a float32 array of shape (len(chunks), EMBED_DIM)

    TODO:
      1. Embed each chunk using embed_text() -> stack into shape (N, EMBED_DIM)
         Hint: np.array([embed_text(c) for c in chunks])
      2. L2-normalise the embeddings in-place:
         faiss.normalize_L2(embeddings)   # modifies in-place
         (after normalisation, inner product = cosine similarity)
      3. Create index = faiss.IndexFlatIP(EMBED_DIM)
      4. Add embeddings to the index: index.add(embeddings)
      5. Return (index, embeddings)
    """
    # YOUR CODE HERE
    embeddings = np.array([embed_text(c) for c in chunks])
    faiss.normalize_L2(embeddings)
    index = faiss.IndexFlatIP(EMBED_DIM)
    index.add(embeddings)
    return index, embeddings
    pass


print('Building index (this takes ~30 seconds on CPU)...')
index, embeddings = build_index(chunks)

# ===== Sanity Check =====
assert index is not None, 'build_index() returned None'
assert index.ntotal == len(chunks), f'Expected {len(chunks)} vectors in index, got {index.ntotal}'
assert embeddings.shape == (len(chunks), EMBED_DIM)
print(f'PASS: Index built! {index.ntotal} vectors of dim {EMBED_DIM}')
```

```
Building index (this takes ~30 seconds on CPU)...
PASS: Index built! 27 vectors of dim 384
```

B4 — Retrieve Relevant Chunks [TODO]

Given a question, embed it and find the most semantically similar chunks.

In [ ]:

```python
def retrieve(
    question: str,
    index: faiss.Index,
    chunks: list[str],
    k: int = 3,
) -> list[tuple[str, float]]:
    """
    Retrieve the top-k most relevant chunks for a question.

    Args:
        question: user's query string
        index: FAISS index containing chunk embeddings
        chunks: original list of chunk strings (same order as index)
        k: number of chunks to retrieve
    Returns:
        list of (chunk_text, similarity_score) tuples, sorted by score descending

    TODO:
      1. Embed the question using embed_text() -> shape (EMBED_DIM,)
      2. Reshape to (1, EMBED_DIM) and convert to float32
      3. L2-normalise: faiss.normalize_L2(query_vec)
      4. Search: scores, indices = index.search(query_vec, k)
         scores and indices are shape (1, k) — take [0] to get 1D arrays
      5. Return [(chunks[idx], score) for idx, score in zip(indices[0], scores[0])]
    """
    # YOUR CODE HERE
    query_vec = embed_text(question)
    query_vec = np.reshape(query_vec, (1, EMBED_DIM)).astype(np.float32)
    faiss.normalize_L2(query_vec)
    scores, indices = index.search(query_vec, k)
    return [(chunks[idx], float(score)) for idx, score in zip(indices[0], scores[0])]
    pass


# ===== Sanity Check =====
test_results = retrieve('What is BERT?', index, chunks, k=3)
assert test_results is not None, 'retrieve() returned None'
assert len(test_results) == 3, f'Expected 3 results, got {len(test_results)}'
assert isinstance(test_results[0][0], str), 'First element of each tuple should be a string'
assert isinstance(test_results[0][1], float), 'Second element of each tuple should be a float'
print('PASS: retrieve() works!')
print('\nTop 3 chunks for "What is BERT?":')
for i, (chunk, score) in enumerate(test_results):
    print(f'\n[{i+1}] Score: {score:.4f}')
    print(textwrap.fill(chunk[:200], width=80))
```

```
PASS: retrieve() works!

Top 3 chunks for "What is BERT?":

[1] Score: 0.6343
BERT (Bidirectional Encoder Representations from Transformers) is an encoder-
only     Transformer introduced by Google in 2018. It is trained using Masked
Language Modelling,     where 15% of input to

[2] Score: 0.5801
age Modelling,     where 15% of input tokens are randomly masked and the model
must predict them using     both left and right context simultaneously. BERT is
primarily used for text classification,

[3] Score: 0.2323
isation) to fine-tune the language     model to maximise the reward model's
scores. ChatGPT and Claude were both trained with RLHF.
```

B5 — Build the RAG Prompt [TODO]

Format the retrieved chunks and the user's question into a prompt the LLM can use.

In [ ]:

```
def build_prompt(question: str, context_chunks: list[tuple[str, float]]) -> str:
    """
    Build an LLM prompt from a question and retrieved context chunks.

    Args:
        question: user's question
        context_chunks: list of (chunk_text, score) tuples from retrieve()
    Returns:
        formatted prompt string

    TODO:
      Build a prompt with this structure:

      Answer the question using only the context below.
      If the answer is not in the context, say "I don't know".

      Context:
      [Source 1]: <chunk text>

      [Source 2]: <chunk text>

      [Source 3]: <chunk text>

      Question: <question>
      Answer:

    Each source should be on its own line, separated by a blank line.
    """
    # YOUR CODE HERE
    prompt = f"Answer the question using only the context below. If the answer is not in the context, say \"I don
't know\".\n\nContext:\n"
    for i, (chunk, score) in enumerate(context_chunks):
        prompt += f"[Source {i+1}]: {chunk}\n\n"
    prompt += f"Question: {question}\nAnswer:"
    return prompt

    pass


# ===== Sanity Check =====
test_chunks_for_prompt = [('Transformers were introduced in 2017.', 0.95),
                          ('BERT is an encoder model.', 0.80)]
prompt = build_prompt('When were Transformers introduced?', test_chunks_for_prompt)
assert prompt is not None, 'build_prompt() returned None'
assert 'Source 1' in prompt, 'Prompt should contain [Source 1]'
assert 'Source 2' in prompt, 'Prompt should contain [Source 2]'
assert 'When were Transformers introduced?' in prompt, 'Prompt must contain the question'
assert 'Answer:' in prompt, 'Prompt must end with Answer:'
print('PASS: build_prompt() works!')
print('\nSample prompt:')
print('-' * 60)
print(prompt)
print('-' * 60)
```

```
PASS: build_prompt() works!

Sample prompt:
------------------------------------------------------------
Answer the question using only the context below. If the answer is not in the context, say "I don't
know".

Context:
[Source 1]: Transformers were introduced in 2017.

[Source 2]: BERT is an encoder model.

Question: When were Transformers introduced?
Answer:
------------------------------------------------------------
```

B6 — Generate an Answer [TODO]

Pass the prompt to the LLM and extract the generated answer.

In [ ]:

```
def generate_answer(prompt: str, max_new_tokens: int = 150) -> str:
    """
    Generate an answer from the LLM given a RAG prompt.

    Args:
        prompt: the formatted RAG prompt from build_prompt()
        max_new_tokens: maximum tokens to generate
    Returns:
        the generated answer string (stripped)

    TODO:
      1. Call generator(prompt, max_new_tokens=max_new_tokens)
         generator returns a list of dicts; get result[0]['generated_text']
      2. Strip whitespace and return the answer string
    """
    # YOUR CODE HERE
    result = generator(prompt, max_new_tokens=max_new_tokens)
    return result[0]['generated_text'].strip()
    pass


# ===== Sanity Check =====
test_prompt = 'Answer in one sentence. What is 2 + 2? Answer:'
answer = generate_answer(test_prompt, max_new_tokens=20)
assert answer is not None, 'generate_answer() returned None'
assert isinstance(answer, str), f'Expected str, got {type(answer)}'
assert len(answer) > 0, 'Answer is empty'
print(f'PASS: generate_answer() works! Sample output: "{answer}"')
```

```
PASS: generate_answer() works! Sample output: "2 + 2"
```

B7 — The Full RAG Pipeline [TODO]

Wire together all the pieces you've built into one function.

In [ ]:

```python
def rag_answer(question: str, index: faiss.Index, chunks: list[str], k: int = 3) -> str:
    """
    Full RAG pipeline: question -> retrieve -> prompt -> generate -> answer.

    TODO:
      1. Call retrieve() to get the top-k chunks
      2. Call build_prompt() with the question and retrieved chunks
      3. Call generate_answer() with the prompt
      4. Return the answer string
    """
    # YOUR CODE HERE
    context_chunks = retrieve(question, index, chunks, k)
    prompt = build_prompt(question, context_chunks)
    answer = generate_answer(prompt)
    return answer
    pass


# ===== Sanity Check =====
test_questions = [
    'What is BERT and what is it used for?',
    'How does RLHF work?',
    'What is the difference between Word2Vec and Transformer embeddings?',
]

print('========== RAG PIPELINE TEST ==========')
for q in test_questions:
    answer = rag_answer(q, index, chunks)
    assert answer is not None and len(answer) > 0, f'Got empty answer for: {q}'
    print(f'\nQ: {q}')
    print(f'A: {textwrap.fill(answer, width=80)}')

print('\nPASS: Full RAG pipeline works!')
```

```
========== RAG PIPELINE TEST ==========

Q: What is BERT and what is it used for?
A: BERT is primarily used for text classification, [Source 3]: RAG (Retrieval-
Augmented Generation) is a technique that combines a language model with an
external knowledge retrieval system. Documents are chunked, embedded using a
sentence encoder, and st

Q: How does RLHF work?
A: It has three stages: supervised fine-tuning on instruction-fol [Source 2]:
isation) to fine-tune the language model to maximise the reward model's scores.
ChatGPT and Claude were both trained with RLHF. [Source 3]: rieved, and these
chunks are injected into the LLM's prompt as context. This allows the model to
answer questions about documents it was never trained on, without the
hallucination risk of rel

Q: What is the difference between Word2Vec and Transformer embeddings?
A: Word2Vec embeddings are static — each word has one fixed vector regardless of
context.

PASS: Full RAG pipeline works!
```

B8 — The Console Chatbot [TODO]

Now build the interactive loop. This is the payoff — put it all together.

In [ ]:

```
def run_chatbot(index: faiss.Index, chunks: list[str]) -> None:
    """
    Run an interactive console chatbot loop.

    TODO:
      Implement a loop that:
      1. Prints a welcome message
      2. Repeatedly:
         a. Prompts the user for input with: question = input('You: ').strip()
         b. If question is empty, continue (ask again)
         c. If question.lower() is 'quit' or 'exit', print a goodbye message and break
         d. Otherwise:
            - Print 'Bot: Thinking...' so the user knows it is working
            - Call rag_answer(question, index, chunks)
            - Print 'Bot: <answer>'
            - Print a blank line for readability

    Example interaction:
    ____________________________________
    ML Chatbot (type 'quit' to exit)

    You: What is a Transformer?
    Bot: Thinking...
    Bot: The Transformer architecture was introduced in 2017...

    You: quit
    Bot: Goodbye!
    ____________________________________
    """
    # YOUR CODE HERE
    print('ML Chatbot (type "quit" to exit)')
    while True:
        question = input('You: ').strip()
        if not question:
            continue
        if question.lower() in ['quit', 'exit']:
            print('Bot: Goodbye!')
            break
        print('Bot: Thinking...')
        answer = rag_answer(question, index, chunks)
        print(f'Bot: {answer}')
        print()

    pass


# Run the chatbot!
# (In Colab, this will create an interactive input box)
run_chatbot(index, chunks)
```

```
ML Chatbot (type "quit" to exit)
You: What is BERT
Bot: Thinking...
Bot: Bidirectional Encoder Representations from Transformers

You: explain in detail
Bot: Thinking...
Bot: I don't know

You: quit
Bot: Goodbye!
```

Part C — The Challenge

Choose **one** of the two extensions below. Both are open-ended — there is no single right answer. You are graded on your approach, code quality, and reflection.

Option 1 — Multi-Turn Memory

Right now, each question is answered independently — the chatbot has no memory of previous turns.

Task: Modify `run_chatbot()` to maintain a conversation history, and include the last N turns in the prompt so the bot can answer follow-up questions coherently.

Example:

```
You: What is BERT?
Bot: BERT is an encoder-only Transformer...

You: What was it trained on?    ← requires memory of previous turn to answer
Bot: BERT was trained using Masked Language Modelling...
```

Hint: Modify `build_prompt()` to accept an optional `history: list[tuple[str,str]]` parameter (list of (question, answer) pairs) and prepend it to the context.

In [ ]:

```
# ===== Option 1: Multi-Turn Memory =====

def build_prompt_with_history(
    question: str,
    context_chunks: list[tuple[str, float]],
    history: list[tuple[str, str]],
    max_history: int = 3,
) -> str:
    """
    Build a RAG prompt that includes recent conversation history.

    TODO:
      Include up to `max_history` recent (question, answer) pairs from history
      in the prompt, before the retrieved context.
      Format them clearly so the model understands they are prior turns.
    """
    # YOUR CODE HERE
    #recent max_history
    recent = history[-max_history:]

    #history
    history_text = "\n".join([f"Question: {q}\nAnswer: {a}" for q, a in recent])

    #context blocks
    context_text = "\n".join([f"[Source {i+1}]: {chunk}" for i, (chunk, score) in enumerate(context_chunks)])

    #combining
    prompt = f"{history_text}\n\nContext:\n{context_text}\n\nQuestion: {question}\nAnswer:"
    return prompt

    pass


def run_chatbot_with_memory(index: faiss.Index, chunks: list[str]) -> None:
    """
    TODO: Modify the chatbot loop to:
      - Maintain a history list of (question, answer) tuples
      - Use build_prompt_with_history() instead of build_prompt()
      - Append each (question, answer) pair to history after each turn
    """
    # YOUR CODE HERE
    history = []
    print('ML Chatbot (type "quit" to exit)')
    while True:
        question = input('You: ').strip()
        if not question:
            continue
        if question.lower() in ['quit', 'exit']:
            print('Bot: Goodbye!')
            break
        print('Bot: Thinking...')
        context_chunks = retrieve(question, index, chunks)
        prompt = build_prompt_with_history(question, context_chunks, history)
        answer = generate_answer(prompt)
        print(f'Bot: {answer}')
        print()
        history.append((question, answer))
    pass


run_chatbot_with_memory(index, chunks)
```

```
ML Chatbot (type "quit" to exit)
You: what is BERT
Bot: Thinking...
Bot: BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only Transformer i
ntroduced by Google in 2018.

You: how is BERT an encoder-only Transformer
Bot: Thinking...
Bot: It is trained using Masked Language Modelling, where 15% of input to [Source 2]: age Modelling,
where 15% of input tokens are randomly masked and the model must predict them using both left and ri
ght context simultaneously. BERT is primarily used for text classification, [Source 3]: GPT (Generat
ive Pre-trained Transformer) is a decoder-only Transformer developed by OpenAI. It is trained on nex
t-token prediction: given a sequence of tokens, predict the next one. GPT models

You: thanks
Bot: Thinking...
Bot: 'Thanks' is a syllable that can be used to describe a given word or phrase.

You: quit
Bot: Goodbye!
```

Option 2 — Source Citation

Right now, the chatbot gives an answer but cites no sources. Users have no way to verify the answer.

Task: After each answer, print the top retrieved source chunks (with their similarity scores) so the user can see *where* the answer came from.

Bonus: Track which document each chunk came from (add a `doc_id` to each chunk during chunking) and display `[Source: Document 3]` alongside each chunk.

In [ ]:

```python
# ===== Option 2: Source Citation =====

def chunk_documents_with_ids(
    documents: list[str],
    chunk_size: int = 200,
    overlap: int = 40,
) -> list[dict]:
    """
    Like chunk_documents(), but returns a list of dicts:
    {'text': str, 'doc_id': int, 'chunk_id': int}

    TODO: Re-implement chunking to track which document each chunk came from.
    """
    # YOUR CODE HERE
    pass


def run_chatbot_with_citations(index: faiss.Index, chunks: list[str]) -> None:
    """
    TODO: Modify the chatbot loop to print the retrieved source chunks
    and their similarity scores after each answer.
    Format them clearly, e.g.:

      Sources:
      [1] (score: 0.87) BERT (Bidirectional Encoder Representations...
      [2] (score: 0.73) The Transformer architecture was introduced...
    """
    # YOUR CODE HERE
    pass


run_chatbot_with_citations(index, chunks)
```

Reflection (Required)

Fill in all five answers before submitting.

1. **Which chunks did your system retrieve for the question "How was ChatGPT trained?"? Were they the right ones? If not, why do you think the retrieval failed?**
   Your answer
2. **Try asking your bot a question whose answer is NOT in the corpus (e.g. 'What is the weather today?'). What does it say? Is this the behaviour you want? How would you fix it?**
   Your answer
3. **What is one concrete limitation of your current chunking strategy? How would you improve it?**
   Your answer
4. **Which Part C option did you implement? Describe your approach in 3–4 sentences and one thing you would do differently with more time.**
   Your answer
5. **You have now built systems using Bag-of-Words (Week 2) and vector embeddings + retrieval (Week 3). In your own words, what is the fundamental difference between how TF-IDF retrieval and embedding-based retrieval find relevant documents?**
   Your answer

Submit your completed notebook to the WnCC submission form. Well done — you've built a RAG chatbot from scratch. 🚀